

Course Project Report

Assembly-Level Execution on Integrated RISC-V Processor

EE/CE 321L/371L – Computer Architecture Lab

Name: _____ ID: _____ Section: _____

Spring 2026

1 Objective

This project brings together everything we have done in the lab so far. The goal is to take our RISC-V assembly programs and actually run them on the single-cycle processor we built in hardware. The project has three parts: first, we run the Lab 10 FSM program on the Lab 11 processor. Then, we add three new instructions that were not in the original processor. Finally, we write and run a Fibonacci program to show the processor works with a real computation.

2 Part A: Base Assembly Program (Lab 10 FSM)

2.1 Overview

For Part A, we took the FSM assembly program from Lab 10 and loaded it into the instruction memory of our Lab 11 processor. The program reads switch values from memory-mapped address 768 (0x300), and when a non-zero value is detected, it calls a countdown subroutine that displays the count on the LEDs at address 512 (0x200), decrementing down to zero. The subroutine uses the stack to save and restore `ra` and `a2`.

2.2 Assembly Program

Listing 1: Lab 10 FSM Program

```
1 # Init
2 addi x28, x0, 5      # test value
3 addi x2,  x0, 511     # stack pointer
4 addi x5,  x0, 768     # switch addr
5 addi x6,  x0, 512     # LED addr
6 sw  x28, 0(x5)
7 sw  x0,  0(x6)        # clear LEDs
8
9 # Wait loop
10 lw   x11, 0(x5)       # read switches
11 beq  x11, x0, -4      # loop if zero
12
13 # Call countdown
14 add  x10, x11, x0     # a0 = switch val
```

```

15 jal x1, 44          # call subroutine
16 beq x0, x0, -20     # jump back to wait
17
18 # Countdown subroutine (PC=44)
19 addi x2, x2, -8      # push
20 sw x1, 4(x2)         # save ra
21 sw x12, 0(x2)        # save a2
22 add x12, x10, x0     # a2 = a0
23 sw x12, 0(x6)        # show on LEDs
24 beq x12, x0, +24     # done if zero
25 addi x12, x12, -1    # decrement
26 addi x13, x0, 3      # small delay
27 addi x13, x13, -1
28 bne x13, x0, -4      # delay loop
29 beq x0, x0, -24      # back to display
30 sw x0, 0(x6)         # clear LEDs
31 lw x12, 0(x2)        # restore a2
32 lw x1, 4(x2)         # restore ra
33 addi x2, x2, 8       # pop
34 jalr x0, 0(x1)       # return

```

2.3 Simulation Results

We ran the simulation in Vivado and set the switches to different values in the testbench. The LEDs showed the correct countdown:

- Switches = 3: LEDs showed $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
- Switches = 5: LEDs showed $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

After each countdown, the LEDs go back to 0 and the processor goes back into the wait loop, polling the switches again. This matches the expected FSM behavior from Lab 5.

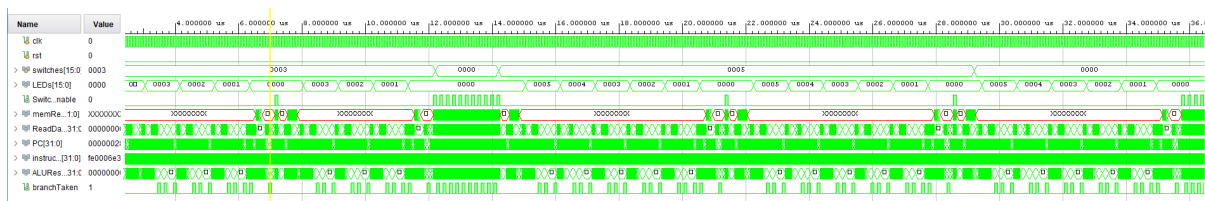


Figure 1: Part A waveform – LED countdown for switch values 3 and 5.

2.4 Note on ALUControl Fix

While testing Part A, we found that the ALUControl encoding from Lab 9 did not match our ALU. For instance, ALUControl was sending 0010 for ADD, but in our ALU 0010 is actually AND. This meant load/store address calculations were doing AND instead of ADD, which gave wrong addresses. We corrected the mappings so they match the ALU properly.

3 Part B: Instruction Extension

We added three new instructions that were not in the original processor. Each one is a different instruction type.

3.1 Instruction 1: SLT (R-type)

SLT (Set Less Than) compares two registers as signed numbers. If $rs1 < rs2$, it writes 1 to rd , otherwise 0. It uses opcode 0110011, $funct3 = 010$, $funct7 = 0000000$.

3.1.1 Changes Made

In **ALU.v**, we added one line – a new case for ALUControl 0111:

```
1 4'b0111: ALUResult = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0; // SLT
```

In **ALUControl.v**, we added the routing so $funct3\ 010$ maps to ALUControl 0111:

```
1 3'b010 : ALUControl = 4'b0111; // SLT
```

3.1.2 How We Tested It

We wrote a small program that does `slt` twice:

- `slt x9, x7, x8` where $x7=5$ and $x8=10 \rightarrow$ result is 1 since $5 < 10$. LEDs showed 0x0001. Correct.
- `slt x9, x8, x7` where $x8=10$ and $x7=5 \rightarrow$ result is 0 since 10 is not less than 5. LEDs showed 0x0000. Correct.

3.2 Instruction 2: BGE (B-type)

BGE (Branch if Greater or Equal) takes a branch if $rs1 \geq rs2$ using signed comparison. It uses opcode 1100011 with $funct3 = 101$.

3.2.1 Changes Made

Only **TopLevelProcessor.v** needed to change. We added one more condition to the `branchTaken` wire. After the ALU does $A - B$, if bit 31 of the result is 0 (meaning the result is not negative), then $A \geq B$:

```
1 wire branchTaken = Branch & (  
2   (funct3 == 3'b000 & Zero)           | // BEQ  
3   (funct3 == 3'b001 & ~Zero)          | // BNE  
4   (funct3 == 3'b101 & ~ALUResult[31]) // BGE  
5 );
```

We did not need to touch ALUControl or MainControl for this – branches already use subtraction, and the branch opcode was already handled.

3.2.2 How We Tested It

We set $x7=10$ and $x8=5$, loaded 0xAA into $x9$, and then ran `bge x7, x8, +8`. Since $10 \geq 5$, it should skip the next instruction (which would overwrite $x9$ with 0xFF) and jump ahead. The LEDs showed 0x00AA, not 0x00FF, so the branch was correctly taken.

3.3 Instruction 3: LUI (U-type)

LUI (Load Upper Immediate) loads a 20-bit immediate into the upper 20 bits of a register, with the lower 12 bits set to zero. Opcode is 0110111.

3.3.1 Changes Made

In **MainControl.v**, we added a new output signal called LUI and a new case:

```
1 7'b0110111: begin // LUI
2     RegWrite = 1'b1;
3     LUI = 1'b1;
4 end
```

In **TopLevelProcessor.v**, we changed the write-back mux so that when LUI is active, the immediate value goes straight to the register file without going through the ALU:

```
1 assign regWriteData = (Jump | JumpReg) ? PCplus4
2                       : (LUI ? imm : writeBackData);
```

We did not need to change **immGen.v** since it already had a case for opcode 0110111 that produces {instruction[31:12], 12'b0}.

3.3.2 How We Tested It

We ran `lui x9, 0xABCDE` which puts 0xABCDE000 into x9. The LEDs show the lower 16 bits, so we expected 0xE000. That is what showed up.

3.4 Part B Results

Test	Instruction	Expected	Observed
SLT (5 < 10)	<code>slt x9, x7, x8</code>	0x0001	0x0001
SLT (10 < 5)	<code>slt x9, x8, x7</code>	0x0000	0x0000
BGE (10 ≥ 5)	<code>bge x7, x8, +8</code>	0x00AA	0x00AA
LUI	<code>lui x9, 0xABCDE</code>	0xE000	0xE000

Table 1: Part B – all three new instructions verified.

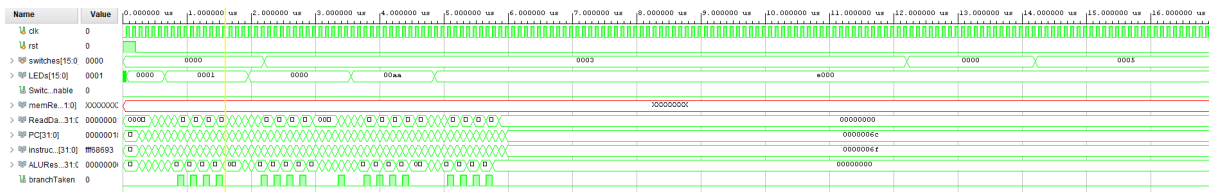


Figure 2: Part B waveform – SLT, BGE, and LUI test results on LEDs.

4 Part C: Fibonacci Program

4.1 Overview

For Part C we wrote a Fibonacci program that computes the first 10 numbers in the sequence and displays each one on the LEDs. We used a subroutine with proper stack

usage for the display and delay logic.

4.2 How It Works

The main loop keeps two variables: `prev` (x7, starts at 0) and `curr` (x8, starts at 1). Each iteration, it displays `curr` on the LEDs by calling a subroutine, then computes `next = prev + curr`, shifts the values (`prev = curr`, `curr = next`), and decrements a counter. After 10 iterations it clears the LEDs and halts.

4.3 Register Usage

Register	Name	What It Holds
x2	sp	Stack pointer (starts at 511)
x6	–	LED address (512)
x7	–	Previous Fibonacci number
x8	–	Current Fibonacci number
x9	–	Temporary for next value
x10	a0	Argument passed to subroutine
x13	–	Delay counter inside subroutine
x14	–	Loop counter (counts down from 10)

Table 2: Registers used in the Fibonacci program.

4.4 Assembly Program

Listing 2: Fibonacci Program

```
1 # Init
2 addi x2, x0, 511      # sp
3 addi x6, x0, 512      # LED addr
4 addi x7, x0, 0        # prev = 0
5 addi x8, x0, 1        # curr = 1
6 addi x14, x0, 10      # count = 10
7
8 # Main loop
9 loop:
10     add x10, x8, x0    # a0 = curr
11     jal x1, display    # call subroutine
12     add x9, x7, x8     # next = prev + curr
13     add x7, x8, x0     # prev = curr
14     add x8, x9, x0     # curr = next
15     addi x14, x14, -1  # count--
16     bne x14, x0, loop  # repeat if not done
17
18 # Done
19     sw x0, 0(x6)       # clear LEDs
20     jal x0, 0          # halt
21
22 # Subroutine: display value on LEDs with delay
23 display:
24     addi x2, x2, -8     # make room on stack
```

```

25      sw    x1, 4(x2)      # save ra
26      sw    x10, 0(x2)    # save a0
27      sw    x10, 0(x6)    # write to LEDs
28      addi   x13, x0, 5    # delay = 5
29 delay:
30      addi   x13, x13, -1
31      bne    x13, x0, delay
32      lw     x10, 0(x2)    # restore a0
33      lw     x1, 4(x2)     # restore ra
34      addi   x2, x2, 8     # pop stack
35      jalr   x0, 0(x1)     # return

```

4.5 Stack Usage

The subroutine saves two registers on the stack: the return address (`ra/x1`) and the argument (`a0/x10`). It decrements `sp` by 8 at the start, stores both, does its work (display + delay), then loads them back and increments `sp` by 8 before returning with `jalr`.

4.6 Simulation Results

The LEDs displayed exactly the Fibonacci sequence:

$$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 13 \rightarrow 21 \rightarrow 34 \rightarrow 55 \rightarrow 0$$

The final 0 means the program cleared the LEDs and entered the halt loop.

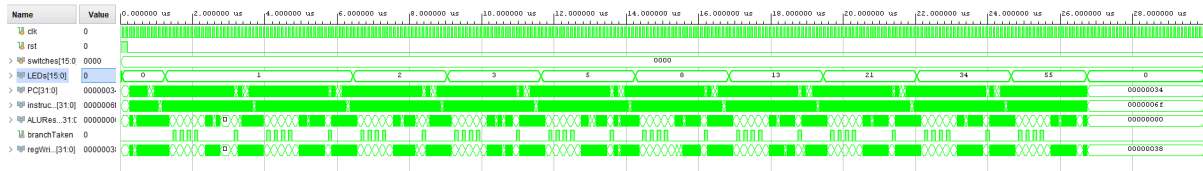


Figure 3: Part C waveform – Fibonacci sequence on LEDs.

5 FPGA Hardware Demonstration

The processor was synthesized and programmed onto the Basys3 FPGA board. All three parts were demonstrated on hardware with the LEDs and switches.

5.1 Part A – FSM Countdown

We loaded the Lab 10 program and flipped the switches to a non-zero value. The LEDs counted down from the switch value to zero, matching the simulation results.

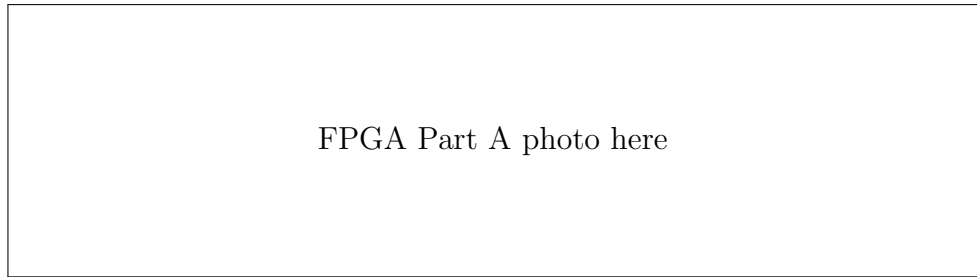


Figure 4: FPGA demo – Part A FSM countdown on LEDs.

5.2 Part B – New Instructions

We loaded the test program for SLT, BGE, and LUI. The LEDs cycled through the expected values (0x0001, 0x0000, 0x00AA, 0xE000) automatically.

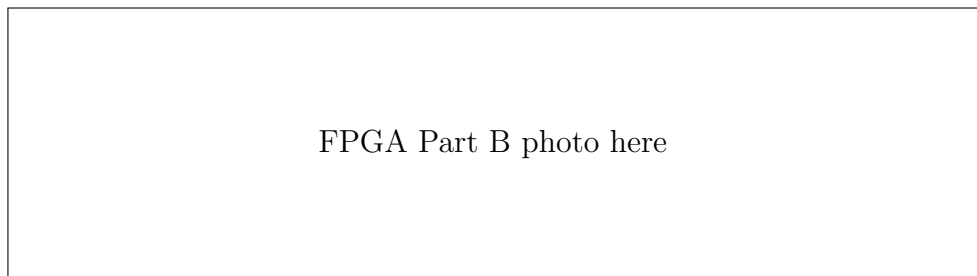


Figure 5: FPGA demo – Part B instruction tests on LEDs.

5.3 Part C – Fibonacci

We loaded the Fibonacci program. The LEDs displayed 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 in sequence before clearing to 0.



Figure 6: FPGA demo – Part C Fibonacci sequence on LEDs.

6 Summary of Changes

The table below lists every file we modified and what we changed. Most changes were just one or two lines.

File	What Changed	Why
ALU.v	Added case 4'b0111	SLT operation
ALUControl.v	Fixed encoding to match ALU	ADD/SUB/AND/OR were swapped
ALUControl.v	Added funct3 010 → 0111	Route SLT to new ALU case
MainControl.v	Added <code>Jump</code> , <code>JumpReg</code> outputs	Needed for JAL/JALR
MainControl.v	Added LUI output + opcode case	New U-type instruction
TopLevelProcessor.v	Added BGE condition	New B-type branch
TopLevelProcessor.v	Added LUI write-back path	Bypass ALU for LUI
instructionMemory.v	Different program per task	Each task has its own program

Table 3: All Verilog modifications across the project.

7 Conclusion

We got all three parts working. Part A runs the Lab 10 FSM countdown on the processor and we verified it with simulation. Part B adds SLT, BGE, and LUI – each a different type (R, B, U) – and all three produce the correct outputs. Part C runs a Fibonacci program that uses a stack-based subroutine, and the LEDs show the right sequence. The main takeaway was understanding how each instruction flows through the datapath, and how small changes to the control logic and ALU can add new capabilities to the processor.